

UNIT 8 · WEEKS 13–14

Networking: sockets

📅 Weeks 13–14 · dates to be confirmed <> Lab TD7 (solved + AI challenge) ~ 2 animations

MANUAL M8 · THEORY OF THE UNIT

Networking: sockets

Core reading for unit 8 (≈ 1 h 30 min). The outputs are real: TD7 programs run on the course machine, over 127.0.0.1.

CONTENTS

- | | | | |
|----|------------------------------------|----|-------------------------------|
| §1 | A socket is a descriptor | §4 | Server models |
| §2 | The life cycle of a TCP connection | §5 | Datagrams and today's network |
| §3 | Addresses, ports and names | | |

§1 A socket is a descriptor

A **socket** is the endpoint of a communication. `socket(domain, type, protocol)` returns a file descriptor, like `open` or `pipe`: it is used with `read`, `write` and `close`, it is inherited through `fork` and it takes an entry of the descriptor table. The *domain* says which addresses it uses: `AF_UNIX` (a path of the file system, on the same machine), `AF_INET` (IPv4) or `AF_INET6` (IPv6). The *type* says how it communicates:

- **Connected** (`SOCK_STREAM`, TCP on the Internet): a connection is set up and then a reliable, ordered **byte stream** travels, as through a pipe. It is the telephone of slide 181.
- **Connectionless** (`SOCK_DGRAM`, UDP): each send is an independent **datagram**, which may be lost, duplicated or arrive out of order. It is the telegram.

The difference can be measured. Three `write`s over TCP arrive in a single `read` —TCP knows nothing about messages—, while over UDP each `recvfrom` returns a whole datagram; and if the receiver does not read in time, UDP loses data without anybody getting an error, even on the same machine:

Real output

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom
#3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992
lost
      kernel counter Udp RcvbufErrors grew by 9992
```

Practical consequence: over TCP, the application defines where each message ends (HTTP/1 uses lines and `Content-Length`; the TD7 echo server, the end of line). And as with pipes, `read` and `write` may be **partial** (unit 2): `write_all` loops until everything is sent.

Practice this → A8.1, scenes 2 and 6 · TD7 · 4

§2 The life cycle of a TCP connection

The **server** creates a socket, gives it an address and a port with `bind` and calls `listen(fd, backlog)`: the socket becomes *passive* (LISTEN state) and only serves to receive connections. The **client** creates its socket and calls `connect` with the server's address. Then the two kernels do the **three-way handshake**: SYN from the client, SYN-ACK from the server, ACK from the client. The key point: **the kernel completes the handshake**, not the application. The connection stays ESTABLISHED in the listening socket's *accept queue*, and `accept` only takes it out of there and returns a **new descriptor** for that client (the listening one keeps accepting others).

The `tcp_states` program plays both roles in one process and asks the kernel for the state of each socket with `getsockopt(TCP_INFO)`:

Real output

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
```

socket+bind+listen(4)	listener LISTEN (queue 0/4) client -	server -
connect (no accept yet)	listener LISTEN (queue 1/4) client ESTABLISHED	server -
accept	listener LISTEN (queue 0/4) client ESTABLISHED	server
ESTABLISHED		
write/read	server read 6 bytes	
client shutdown (FIN)	listener LISTEN (queue 0/4) client FIN_WAIT2	server
CLOSE_WAIT		
server read	server read returns 0: end of file	
server shutdown (FIN)	listener LISTEN (queue 0/4) client CLOSED	server
CLOSED		
	the client's end moved to a TIME_WAIT entry of the kernel	
(60 s, see ss)		
Recv-Q	Send-Q	Local Address:Port Peer Address:Port
0	0	127.0.0.1:42772 127.0.0.1:PORT

Closing is symmetric and in two halves. Whoever calls `close` first (or `shutdown(fd, SHUT_WR)`) sends FIN and goes through `FIN_WAIT1` and `FIN_WAIT2`; the other side receives the FIN, stays in `CLOSE_WAIT` and its `read` returns 0 —end of file, as with a pipe without writers—. When it closes in turn (`LAST_ACK`) and the last ACK arrives, the first side stays in **TIME_WAIT** for 2·MSL (60 s in Linux), in case the last ACK was lost and the FIN is repeated. That entry outlives the program (`ss` shows it) and is the reason for `SO_REUSEADDR`: without it, restarting a server fails with `EADDRINUSE`.

The backlog does not limit how many clients the server serves, but how many already established connections can wait for `accept`. Linux admits `backlog + 1`; with the queue full it

drops the SYN and the client resends it with exponential backoff (1 s, 2 s, 4 s...):

Real output

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```



For the curious: SYN cookies

With a very small backlog (`./backlog 2 8`) the kernel ends up answering with *SYN cookies*, a defence against SYN floods that keeps no state. If the final ACK arrives with the accept queue full, it is lost: the client believes it is connected and the server does not know about it. The `nstat` counters show it in TD7 · 4. That is why real servers ask for a large backlog (the cap is `net.core.somaxconn`).

Practice this → A8.1 life cycle of a connection · TD7 · 1 and 4

§3 Addresses, ports and names

Each domain has its address structure — `sockaddr_un` with a path of up to 108 bytes, `sockaddr_in` with an IPv4 address and a port, `sockaddr_in6` for IPv6— and all are passed to `bind`, `connect` or `accept` as `struct sockaddr *` together with their size; `struct sockaddr_storage` is big enough for any. The numbers inside the structure are in **network order** (*big-endian*): hence `htons` for the port and `htonl` for the address. The example on slide 187 forgot it, and the bug can be seen:

Real output

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:      the kernel bound port 22555
sin_port = htons(7000): the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):  bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110, sockaddr_storage
128
```

On an x86-64, 7000 = 0x1B58 is stored as the bytes 58 1B; read as *big-endian* they are 0x581B = 22555, and the server ends up listening on another port. The same output shows three more things: `getaddrinfo` turns a name and a service (from `/etc/hosts` and `/etc/services`, or from

DNS) into a list of IPv4 and IPv6 addresses ready to use —it replaces `gethostbyname`, which only knew IPv4, was not thread-safe and left POSIX in 2008—; ports below 1024 require privileges (`CAP_NET_BIND_SERVICE`); and the real sizes of the structures.

An `AF_UNIX` socket uses neither IP nor ports: its address is a **file of type socket**. Besides, the kernel tells the server who is on the other side (`SO_PEERCRED`: PID and UID), something systemd, D-Bus or `/var/run/docker.sock` use to decide what to allow:

Real output

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108
bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

Practice this → TD7 · 4, parts 5 and 6

§4 Server models

A server spends most of its time **waiting**: for a client to arrive (`accept`) or for that client to send something (`read`). How it organizes those waits decides who waits for whom:

- **Iterative**: a single thread serves one client at a time. If a client is slow, everybody else waits in the queue.
- **A process per client** (`fork` after `accept`): the child serves and the parent goes back to `accept`. Isolated from each other, but creating a process costs several times more than a thread (measured in TD6 · 5) and zombies must be avoided.
- **A thread per client**: cheaper and with shared memory, but 10,000 connections would be 10,000 threads with their stacks.
- **Thread pool**: k threads created at start consume connections from a bounded queue filled by the main thread —the producer–consumer of unit 7—. Bounded resources, but k slow clients saturate it.
- **Event loop**: one thread asks the kernel with `epoll_wait` which sockets already have data and only serves those, so it never blocks on a slow client. It is the answer to the C10K problem and the model of nginx or Node.js; `io_uring` (Linux 5.1) takes the idea further.

TD7 implements all five in `http_server.c` and measures them with 32 clients, first 1 and then 8 that take 1 s to send their request:

Real output

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 982 ms, max 983 ms
```

Real output

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 981 ms, max 981 ms
```

Real output

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
  slow 1: median 1001 ms, max 1001 ms
  fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
  slow 8: median 1001 ms, max 1001 ms
  fast 24: median 1 ms, max 2 ms
```

Practice this → A8.2 server models · TD7 · 3 and 4

§5 Datagrams and today's network

In connectionless mode there is no `listen` or `accept`: the server binds a well-known port and waits with `recvfrom`, which returns a datagram and the address of whoever sent it; the client uses `sendto` with the destination address (slides 194–197). A UDP socket can also be `connect`ed to fix the destination and use `send` / `recv`. UDP is the basis of DNS, of real-time voice and video and, since 2021, of **QUIC** (RFC 9000), on which **HTTP/3** (RFC 9114) runs: reliability and encryption are implemented on top, in the program and not in the kernel.

Two more changes with respect to the original course. Almost all traffic is encrypted with **TLS** (https, 443): the socket carries encrypted bytes and a library (OpenSSL, GnuTLS...) does the work.

And the unencrypted services of slide 183 (`telnet` , `ftp`) were replaced by `ssh` and `https`.

Practice this → A8.1, scene 6 (UDP) · TD7 · 4